

CivicPulse — SIH Presentation Pack

Everything the team needs to build the deck. Each section below is **one slide**. Text in `>` blocks is speaker notes — say it, don't put it on the slide.

Live demo: <https://civicpulse-black.vercel.app> **Code:** <https://github.com/lkjotsingh2710/CivicPulse>

Design rule for the whole deck: judges read slides in about eight seconds. Put the headline and a diagram on the slide; keep the sentences for the mouth. Never paste a paragraph from this file onto a slide — pull the bold phrases.

Slide 1 — Title

CivicPulse *Report it once. Track it to resolved.*

- Problem Statement ID: `<fill in>`
- Problem Statement Title: `<fill in>`
- Theme: Clean & Green Technology / Smart Automation
- Category: Software
- Team Name: `<fill in>` · Team ID: `<fill in>`

Open with one sentence, not a feature list: *"Everyone has reported a pothole and watched nothing happen. We built the system that makes ignoring it hard."*

Slide 2 — The Problem

Civic complaints disappear into a queue nobody prioritises.

Four failures, every existing system has all four:

#	Failure	What it costs
1	No proof of location	Complaints get filed from a sofa. Crews cannot find the spot
2	Newest first	A pothole affecting 50 people sinks below one affecting nobody
3	Duplicates	One problem filed 5 times = the desk works it 5 times
4	No feedback loop	The citizen never learns what happened, so they never report again

The fourth is the one that kills these systems. People stop reporting not because reporting is hard, but because last time nothing visibly happened. Every design decision we made comes back to closing that loop.

On campuses specifically: thousands of students live with broken streetlights and overflowing bins, and have no route to the maintenance desk at all — just a WhatsApp group that nobody in charge reads.

Slide 3 — Proposed Solution

One report per problem. Everyone else adds weight. Fixing it earns rewards.

Four mechanisms:

- Live camera only.** No gallery uploads. The photo is taken through the app and geotagged at the instant of capture — **you must physically be at the problem to report it.**
- Upvote, don't duplicate.** If you try to file something already reported within 75m, the app shows you that report and asks "same problem?" Yes adds your weight; no files normally. The ward desk queue sorts by **weight, not recency** — so a problem affecting fifty people rises instead of sinking.
- Honest closure.** Every report ends as *Resolved with a proof photo* or *Rejected with a reason the citizen reads*. There is no third outcome where it quietly rots.
- Pulse Points.** 5 points when the ward desk **approves** your campus report — redeemable for vouchers at 50 and 100 points.

The innovation is not the app. It's paying on **validation, not submission**. Every rewarded reporting app gets farmed within a week because it pays on upload — so the spam filter has to be perfect, and no filter is. Ours pays only when a human accepts the report, so junk earns nothing no matter what slips through. That one decision replaces an entire fraud-detection system.

Slide 4 — Technical Approach

Angular 22 SPA → Supabase (Postgres) · no application server



Layer	Technology	Why this one
Frontend	Angular 22, signals, lazy routes	Every page loads on demand
Backend	Supabase — Postgres + PostgREST + GoTrue	Free tier; RLS makes serverless safe
Auth	Phone + password, no OTP	SMS costs nothing because we send none
Geo	Browser Geolocation + haversine in SQL	No PostGIS, no maps API key
Hosting	Vercel	Free HTTPS — which the camera API requires

Scale: 76 files · ~9,000 lines TypeScript · ~1,250 lines SQL · 13 tests

If a judge asks "why no backend server?" — because there is nothing for it to do. Postgres already authenticates,

authorises and validates. Adding a Node layer would mean a second place for rules to live and disagree.

Slide 5 — Security: the browser is not trusted

Every rule that matters is a database policy, not JavaScript.

The only credential in the browser is Supabase's *publishable* key — public by design. Anyone can open DevTools and issue any request. The system survives that.

We tested it as an attacker, with only that key:

Attack attempted	Result
Insert 9,999 points into the ledger	✗ violates row-level security policy
Redeem a reward while signed out	✗ Sign in before redeeming a reward
Read another citizen's points ledger	✗ 0 rows
Read the voucher-code pool	✗ 0 rows
List every report photo in storage	✗ blocked

That last row was a real bug we found and fixed. The original storage policy allowed *listing* the bucket, not just fetching a file you had a link to — so anyone could have downloaded every report photo ever uploaded. Migration `0006` closes it. **Say this out loud.** Judges trust a team that found and names its own vulnerability far more than one that claims it had none.

Passwords: never stored by us. They go straight to Supabase Auth, which bcrypt-hashes them. No table in our schema has a password column.

Slide 6 — Anti-gaming design

A reward system is an adversarial system. We designed for that.

Attack	Defence	Where enforced
File junk, collect points	Points only on desk approval	Trigger
Report a city pothole as campus	25-campus whitelist table	Trigger
File the same pothole 20×	Duplicate detection + one payout per ticket	Partial unique index
Upvote-farm your own report	Cannot upvote your own, or twice	Trigger + unique constraint
File a ticket pre-marked "Resolved"	Every citizen report forced to <code>Submitted</code>	Trigger
Redeem 50 points twice at once	Advisory lock + single transaction	<code>redeem_reward()</code>

The double-redemption one is worth calling out — two taps fired together would both read a balance of 50 and both spend it. A per-user advisory lock inside the transaction makes that impossible. This is the kind of detail that separates a demo from something you could actually run.

Upvotes deliberately earn nothing. Anything that paid for upvoting would be farmed in seconds by tapping down a list.

Slide 7 — Feasibility & Viability

Feasibility: it is built, deployed and publicly reachable right now.

Not a mockup — a working system on free infrastructure.

Concern	Reality
Cost to run	₹0. Supabase free tier, Vercel Hobby, no SMS, no maps key, no AI API
Scale	Postgres handles the load; only the free-tier row cap binds first
Adoption	Zero training — snap, pick a category, done
Deployment	<code>git push</code> → live in 90 seconds

Challenges we identified, and our answers:

Risk	Strategy
Wrong-subject photos (selfies, blank walls)	Human rejection workflow with reasons — already built. Automated screening designed but deliberately unbuilt: it costs per report and a false rejection is worse than a junk ticket
False duplicate matches	We ask instead of blocking. Two real potholes sit 40m apart; a silent block would lose genuine reports
No password recovery	Known gap. The synthetic email domain cannot receive mail — next on the roadmap
Ward desks ignoring the queue	24-hour auto-escalation to the department

Do not hide the gaps. Judges probe. A team that says "*here is what we haven't built and why*" reads as engineers; a team that claims everything works gets caught on the one thing that doesn't.

Slide 8 — Impact & Benefits

Target users: students and residents · ward and campus maintenance desks · municipal departments

Dimension	Impact
Social	Closes the feedback loop — proof photos and honest rejections mean people keep reporting
Civic	Desks work the problems affecting the most people first, not the newest
Economic	Duplicate merging removes repeated dispatch for one problem. ₹0 running cost
Behavioural	Rewards make civic participation a habit for students, not a one-off
Environmental	Faster waste and water-leak response; less standing water and less waste

Measurable outcomes once deployed on a campus:

- Median time from report → resolution
- % of reports closed with proof
- Duplicate rate before vs. after merging
- Repeat-reporter rate — *the real signal that the loop is closed*

If you only quote one metric, quote the last one. Every civic app can show reports filed. Almost none can show people **coming back**.

Slide 9 — What's built vs. roadmap

Working today

✓ Phone + password auth, no OTP · ✓ Live camera + geotag capture · ✓ Full report workflow with proof photos · ✓ Rejection with citizen-visible reasons · ✓ Ward control desk (reports / feedback / citizens, CSV export) · ✓ Public regional feed by city or pincode · ✓ Upvoting with priority sorting · ✓ Duplicate detection and merging · ✓ Pulse Points ledger and atomic redemption · ✓ Escalation countdown · ✓ Deployed on HTTPS

Roadmap

→
SOON Password recovery (admin-side reset) · →
SOON Escalation emails (Edge Function) · →
SOON Voucher partnerships to stock the code pool · →
SOON Perceptual-hash duplicate detection (free, catches re-photographed screens) · →
SOON Trust tiers — auto-approve proven reporters, review new accounts · →
SOON Municipal API integration

Framed this way the roadmap is a strength: it shows you know what "finished" means and have sequenced it by value, not by what was easy.

Slide 10 — References

- Swachhata-MoHUA — national grievance redressal app (baseline; no community weighting, no duplicate merging)
- MyGov / CPGRAMS — central public grievance portals
- Supabase Row-Level Security documentation — the authorisation model
- OWASP ASVS — the "never trust the client" principle this architecture follows
- Haversine formula — geodesic distance used for duplicate detection

Demo script — 3 minutes

Rehearse this. Do not explore live.

#	Time	Action	Say
1	0:00	Open the live site on a phone	"This is running in production right now"
2	0:20	Pick campus + category, tap the shutter	"Live camera only — no gallery upload, so you must be standing at the problem"
3	0:45	Allow location, file the report	"Geotagged at capture, not typed in"
4	1:05	Second device: file the same category nearby	"Watch what happens to a duplicate"
5	1:20	The <i>Already reported?</i> prompt appears → tap Yes	"It asks rather than blocks — two real potholes can be 40m apart. This adds weight instead of creating a second ticket"
6	1:45	Sign in as admin → <code>/admin</code>	"The desk sees it ranked by people affected, not by date"
7	2:10	Move it to <i>In Progress</i>	"That approval is what pays the reporter"
8	2:25	Back on the citizen device → Pulse Points	"5 points, credited by the database, not the app"
9	2:45	Show the ledger entry	"Every point is an auditable row. There is no balance we can silently inflate"

Have ready before you start: two devices signed in as different users, one report already approved so the points screen isn't empty, and one rejected report to show the reason the citizen sees.

If the network dies: screen-record the whole flow beforehand and keep it on the laptop. Never demo live without a recorded fallback.

Anticipated questions

"What stops someone photographing the same pothole from ten accounts?"

Each account needs a distinct phone number, points only pay when a human approves, and the duplicate check merges nearby reports of the same category. Ten accounts get one approval and one payout between them.

"Why not use AI to check the photo matches the category?"

We designed it and chose not to build it yet. It costs money per report, and false rejections are worse than junk tickets — "Broken Streetlight" reports are photos of darkness, exactly what a naive classifier rejects. The human rejection workflow handles it today. When we add AI it will *flag for review*, never auto-reject.

"Isn't the API key in the browser a security hole?"

No — it's the publishable key, public by design. Authorisation is Row-Level Security in Postgres. We tested it as an attacker with that key; forging points and reading other users' data are both refused by the database. (*Slide 5.*)

"How is this different from the Swachhata app?"

Three things it doesn't have: community weighting so severity is visible, duplicate merging so one problem is one ticket, and a reward loop that pays on verified resolution. Plus we're campus-first, where there's currently no route to the maintenance desk at all.

"What if a ward desk just ignores everything?"

Every unresolved ticket runs a 24-hour clock and escalates to the department. Citizens can see that countdown, so inaction is visible rather than silent.

"How much does it cost to run?"

Nothing. Supabase free tier, Vercel free tier, no SMS because we don't use OTP, no maps API key, no AI calls. That's a deliberate constraint, not an accident — it means a campus can adopt it with zero budget approval.

Slide-building checklist

- ☐ Fill in Problem Statement ID, title, team name and ID on Slide 1
- ☐ Rebuild the ASCII diagrams (Slides 4, and the tables on 5–6) as clean graphics
- ☐ Add 3–4 real screenshots: the capsule, the duplicate prompt, the admin queue with upvote badges, the Pulse Points wallet
- ☐ Add a QR code to the live URL — let judges open it on their own phones
- ☐ Keep to ~6 lines of text per slide; the detail lives in your mouth
- ☐ Practise the demo end to end at least three times
- ☐ Record the demo as a fallback video